

Day01_Report

Target description

Local VM, IP: 10.0.2.15

Nmap commands used.

nmap --Version

```
ubuntu@ubuntu-2204:~$ nmap --version
Nmap version 7.80 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.3.6 openssl-3.0.2 nmap-libssh2-1.8.2 libz-1.2.11 libpcap-1.10.1 nmap-libdnet-1.12 ipv6
Compiled without:
Available nsock engines: epoll poll select
```

nmap -sS 10.0.2.15 -oN syn_scan.txt

```
ubuntu@ubuntu-2204:~$ sudo -i
root@ubuntu-2204:~# nmap -sS 10.0.2.15 -oN syn_scan.txt
Starting Nmap 7.80 ( https://nmap.org ) at 2025-08-18 12:35 EDT
Nmap scan report for 10.0.2.15 (10.0.2.15)
Host is up (0.0000060s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds
```

nmap -sT 10.0.2.15 -oN tcp_scan.txt

```
root@ubuntu-2204:~# nmap -sT 10.0.2.15 -oN tcp_scan.txt
Starting Nmap 7.80 ( https://nmap.org ) at 2025-08-18 12:37 EDT
Nmap scan report for 10.0.2.15 (10.0.2.15)
Host is up (0.00014s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
Nmap done: 1 IP address (1 host up) scanned in 0.21 seconds
```

nmap -sU 10.0.2.15 -oN udp_scan.txt

```
root@ubuntu-2204:~# nmap -sU 10.0.2.15 -oN udp_scan.txt
Starting Nmap 7.80 ( https://nmap.org ) at 2025-08-18 12:39 EDT
Nmap scan report for 10.0.2.15 (10.0.2.15)
Host is up (0.0000060s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
5353/udp  open|filtered zeroconf
Nmap done: 1 IP address (1 host up) scanned in 1.33 seconds
```

Table of findings:

Port	Service/Protocol	Common Uses	Vulnerabilities	Recommendations
22	SSH (TCP)	Remote login, SCP	Brute-force, weak credentials, leaked keys	Strong auth, restrict access
5353	mDNS (UDP)	Discovery, Bonjour	DDoS, network enumeration, MitM, rebinding	Limit to LAN, firewall controls

Security risks per finding.

(1) Published: 2023-09-27

Updated: 2025-03-25

Title: Non-MFA account takeover via using only SSH public key to login in jumpserver

Description

JumpServer is an open-source bastion host. As an unauthenticated user, it is possible to authenticate to the core API with a username and an SSH public key without needing a password or the corresponding SSH private key. An SSH public key should be considered public knowledge and should not be used as an authentication secret alone. JumpServer provides an API for the KoKo component to validate user private key logins. This API does not verify the source of requests and will generate a personal authentication token. Given that public keys can be easily leaked, an attacker can exploit the leaked public key and username to authenticate, subsequently gaining access to the current user's information and authorized actions. This issue has been addressed in versions 2.28.20 and 3.7.1. Users are advised to upgrade. There are no known workarounds for this vulnerability.

CWE 1 Total

[CWE-862: CWE-862: Missing Authorization](#)

CVSS 1 Total

8.2	HIGH	3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
-----	------	-----	--

Product Status

[Learn more](#)

Vendor

jumpserver

Product

jumpserver

Versions 2 Total

Default Status: *unknown*

affected

affected at **>= 2.0.0, < 2.28.20**

affected at **>= 3.0.0, < 3.7.1**

References 2 Total

github.com: <https://github.com/jumpserver/jumpserver/security/advisories/GHSA-fr8h-xh5x-r8g9>

sonarsource.com: <https://www.sonarsource.com/blog/diving-into-jumpserver-attackers-gateway-to-internal-networks-1-2>

(2) Published: 2024-11-04

Updated: 2024-11-04

Title: Failure to check for packets from the broadcast address allows potential DDoS amplification attack in Meshtastic firmware

Description

Meshtastic firmware is a device firmware for the Meshtastic project. The Meshtastic firmware does not check for packets claiming to be from the special broadcast address (0xFFFFFFFF) which could result in unexpected behavior and potential for DDoS attacks on the network. A malicious actor could craft a packet to be from that address which would result in an amplification of this one message into every node on the network sending multiple messages. Such an attack could result in degraded network performance for all users as the available bandwidth is consumed. This issue has been addressed in release version 2.5.6. All users are advised to upgrade. There are no known workarounds for this vulnerability.

CWE 2 Total

- [CWE-138: CWE-138: Improper Neutralization of Special Elements](#)
- [CWE-159: CWE-159: Improper Handling of Invalid Use of Special Elements](#)

CVSS 1 Total

5.3	MEDIUM	3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L
-----	--------	-----	--

Product Status

[Learn more](#)

Vendor

meshtastic

Product

firmware

Versions 1 Total

Default Status: unknown

affected

- affected at < 2.5.6

References 1 Total

- [github.com: https://github.com/meshtastic/firmware/security/advisories/GHSA-xfmq-5j3j-vgv8](https://github.com/meshtastic/firmware/security/advisories/GHSA-xfmq-5j3j-vgv8)
-

Research on two services.

1. Port 22 (SSH)

- Service Purpose:

- SSH (Secure Shell) runs on port 22 and provides encrypted remote login, command execution, and file transfer capabilities for users and administrators.
- SSH is the backbone for secure device management in Linux, Unix, and network appliances.

- Recent Vulnerabilities (2025):

- Critical RCE in Erlang/OTP SSH (CVE-2025-32433):
 - Severity: CVSS 10.0 (Critical).
 - Issue: Unauthenticated remote code execution due to a flaw in Erlang/OTP's SSH library—attackers can exploit this without any credentials.
 - Affected: Erlang/OTP SSH prior to versions OTP-27.3.3, OTP-26.2.5.11, and OTP-25.3.2.20.
 - Risk: Full system takeover, especially in IoT, OT, and telecom infrastructure using Erlang-based SSH.
 - Fix: Patch to a safe Erlang/OTP release or block SSH access from untrusted networks.
- OpenSSH Vulnerabilities (CVE-2025-26465, CVE-2025-26466):

- MitM attack risk if VerifyHostKeyDNS is enabled.
 - Pre-auth denial-of-service attacks on certain OpenSSH versions (fixed in OpenSSH 9.9p2).
 - General Risks:
 - Brute force login attempts.
 - Use of weak credentials.
 - Exposed root accounts.
- Common Mitigations:
- Keep SSH software up-to-date.
 - Use key-based (not password-based) authentication.
 - Disable root login.
 - Restrict SSH port access by IP (firewall).
 - Monitor for suspicious access attempts.
-

2. Port 5353 (mDNS/Bonjour/ZeroConf)

- Service Purpose:
- mDNS (Multicast DNS) on port 5353 is used for device/service discovery on local networks (no central DNS server needed).
 - Commonly used by Apple's Bonjour, Linux's Avahi, IoT gadgets, printers, and smart home hardware.
- Known Vulnerabilities and Risks:
- DDoS Amplification: Open or misconfigured mDNS can be leveraged for distributed denial-of-service attacks against network targets.
 - Network Enumeration: Attackers can query mDNS to identify devices and services available, which aids lateral movement.
 - MitM Potential: Responses are unencrypted—an attacker inside the network may spoof or intercept traffic.
 - Exposure Risk: Port 5353 should not be exposed to the public Internet; it is meant to stay within the LAN. Exposing this can make local resources available to attackers.
- Common Mitigations:
- Limit mDNS traffic to local subnets only.
 - Block UDP port 5353 at network gateways.

- Regularly monitor for unexpected or unauthorized mDNS broadcasts or responses.
- Place IoT and printers on dedicated VLANs if possible.

Port	Service	Purpose	Common Vulnerabilities	Security Recommendations
22	SSH	Remote admin, secure commands	CVE-2025-32433 (Erlang), CVE-2025-26465/26466 (OpenSSH), brute force, root exposure	Patch promptly, strong auth, restrict access
5353	mDNS	LAN device discovery	DDoS, enumeration, MitM, exposure	Limit LAN use, firewall, VLAN segmentation

- SSH (port 22) can be vulnerable to unauthenticated remote execution if outdated, and risks increase with global exposure and weak credentials.
- mDNS (port 5353) offers convenient device discovery but should be carefully managed, as it's susceptible to DDoS and internal network mapping if exposed beyond the local segment.

Key learnings.

Understanding Network Fundamentals:

I gained clarity on essentials like IP addresses, ports, and protocols (TCP/UDP), which underpin how services communicate over networks.

Proficiency with Nmap Scanning:

I learned to use different Nmap scan types—SYN scan, TCP connect scan, UDP scan—and how to interpret their outputs. This included identifying open ports, associated services, and optional OS fingerprinting.

Security Implications of Open Ports:

Analyzing the scan results highlighted the importance of managing open ports carefully. For example, SSH on port 22, while critical for secure remote management, needs strong security controls to prevent credential theft or exploitation of vulnerabilities like the recent critical CVE-2025-32433. Similarly, the mDNS service on port 5353, commonly used for device discovery, must be restricted to local networks to avoid risk from DDoS amplification or network enumeration attacks.

Researching Service Vulnerabilities:

Investigating specific vulnerabilities and risks gave me practical knowledge of real-world threats, such as brute-force attacks on SSH, risks of weak authentication, and abuse potential of multicast DNS services.

Importance of Defense-in-Depth:

Securing network services requires multiple layers—strong authentication, firewall rules, patch management, traffic segmentation, and continuous monitoring.

Automation with Python Scripting:

Writing a Python script to automate Nmap scans reinforced my scripting skills and demonstrated how automation can streamline security assessments and reporting.

Effective Communication through Reporting and Visualization:

Creating a detailed report and network diagram helped synthesize technical findings into understandable formats, which is essential for cybersecurity documentation and collaboration.

Diagram (as link or embedded image).

